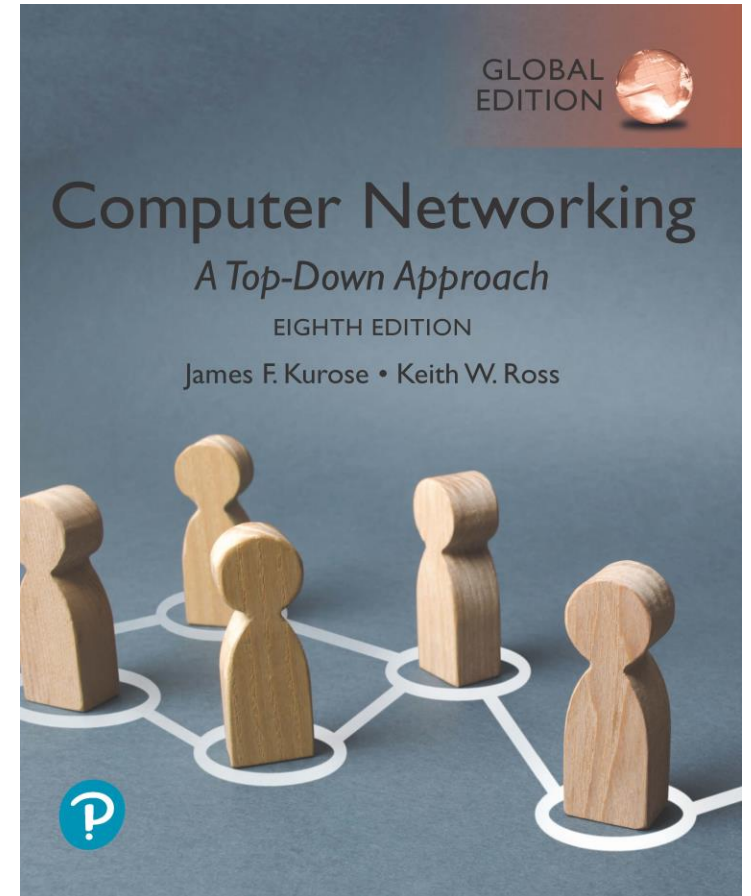


Chapter 3

Transport Layer



Computer Networking: A Top-Down Approach

8th Edition, Global Edition

Jim Kurose, Keith Ross

Copyright © 2022 Pearson Education Ltd

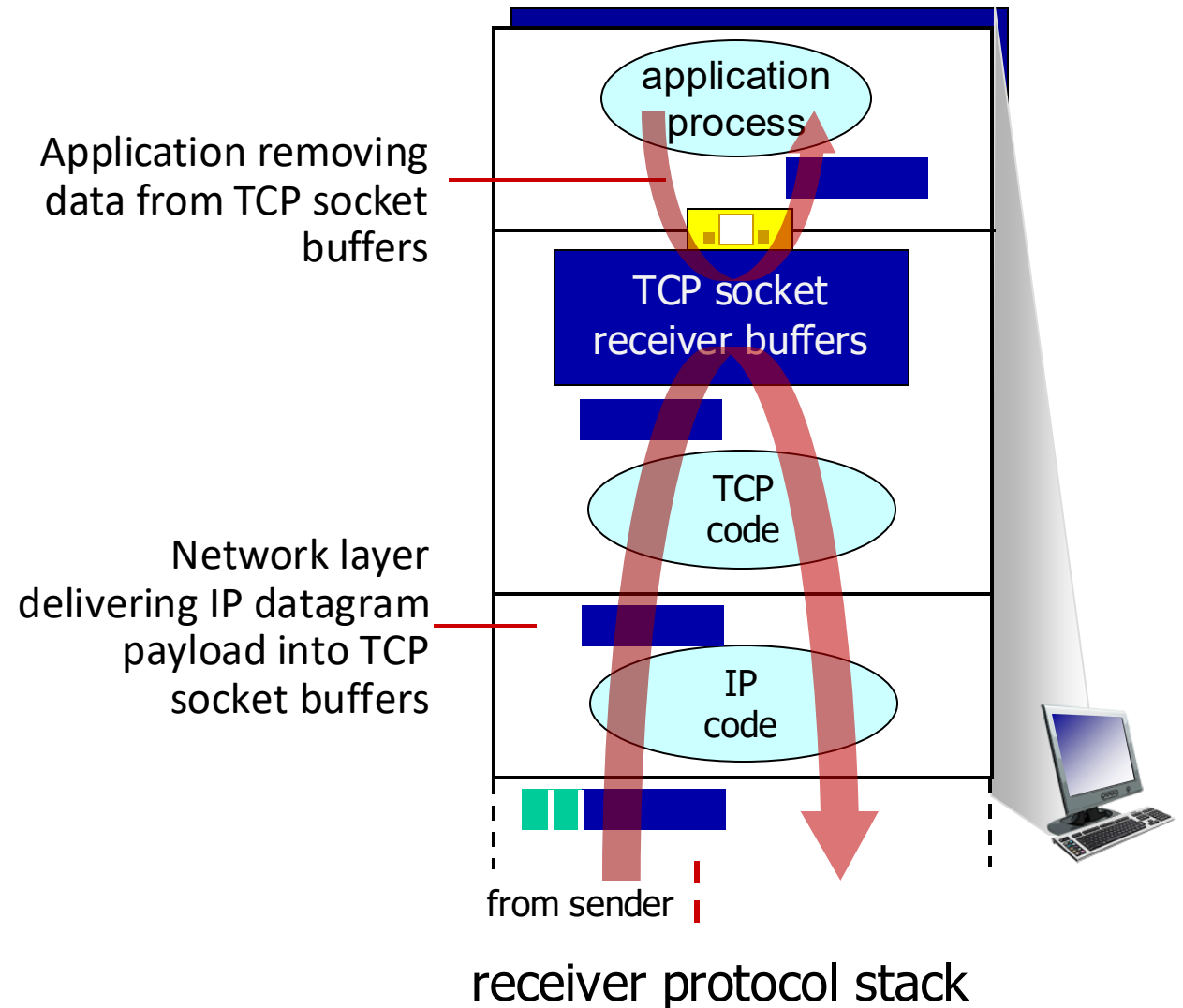
Chapter 3: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- **Connection-oriented transport: TCP**
 - segment structure
 - reliable data transfer
 - flow control
 - connection management
- Principles of congestion control
- TCP congestion control



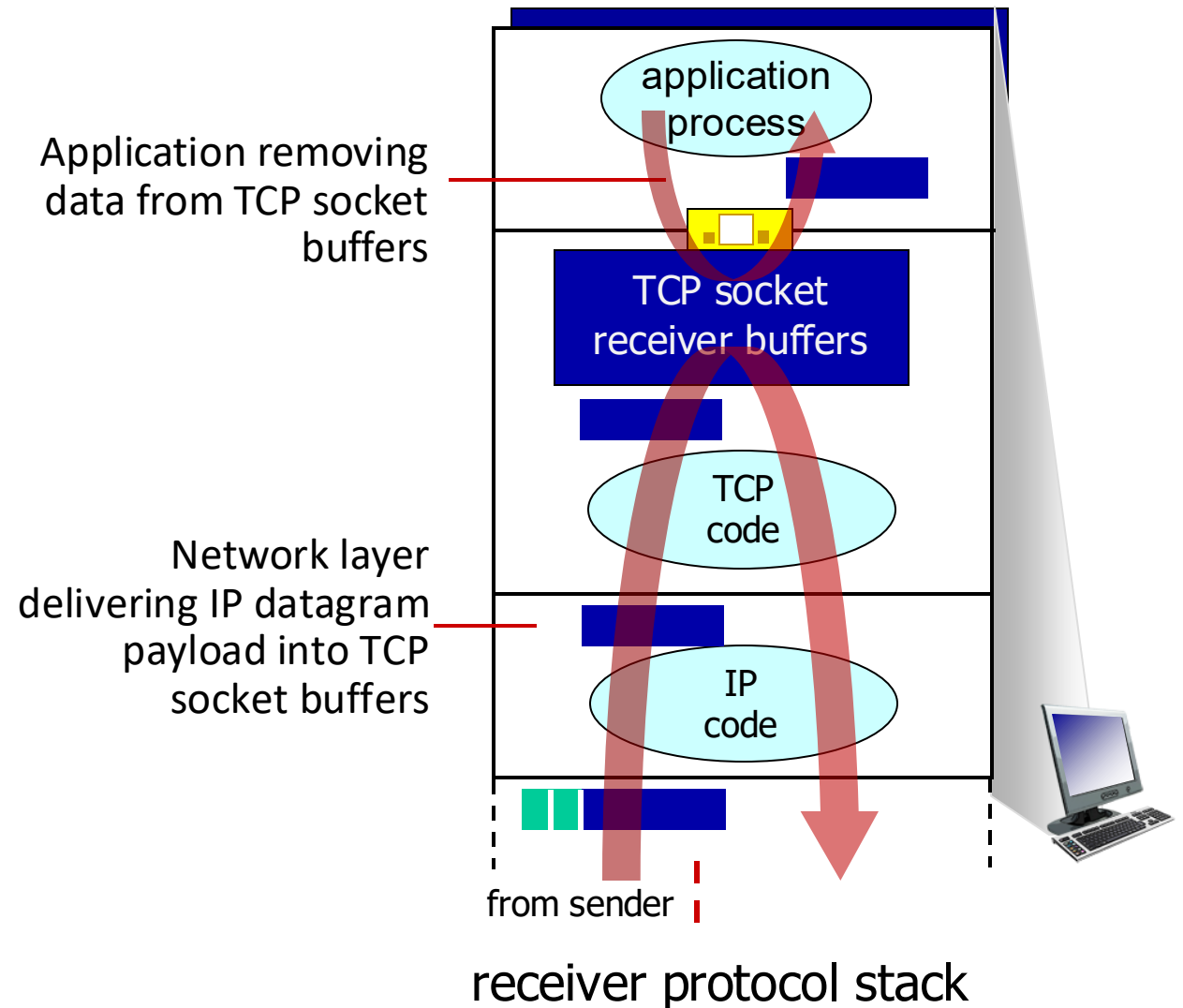
TCP flow control

Q: What happens if network layer delivers data faster than application layer removes data from socket buffers?



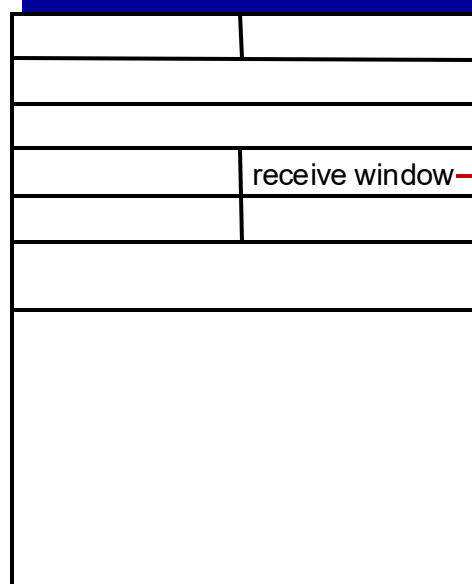
TCP flow control

Q: What happens if network layer delivers data faster than application layer removes data from socket buffers?

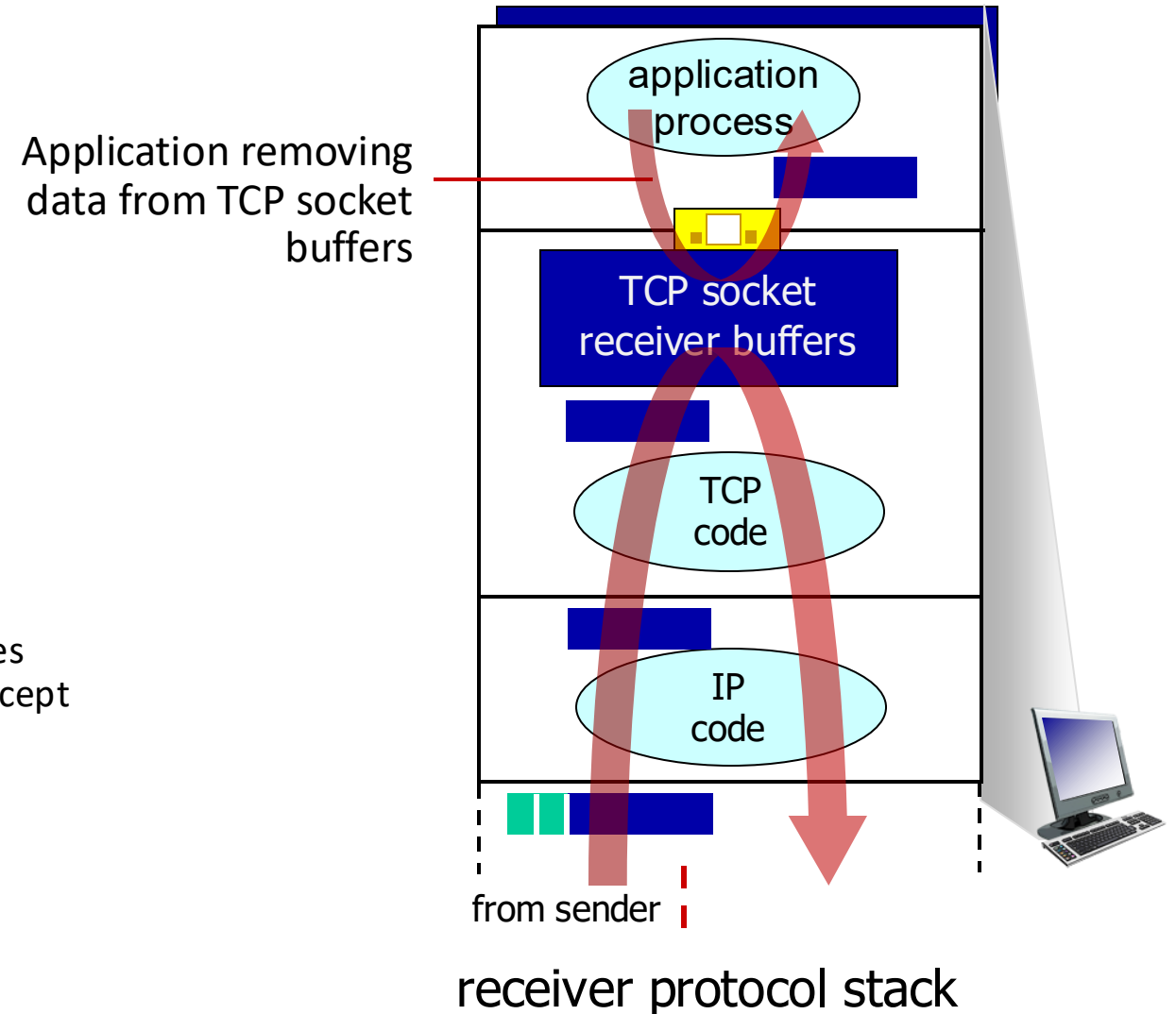


TCP flow control

Q: What happens if network layer delivers data faster than application layer removes data from socket buffers?



flow control: # bytes
receiver willing to accept

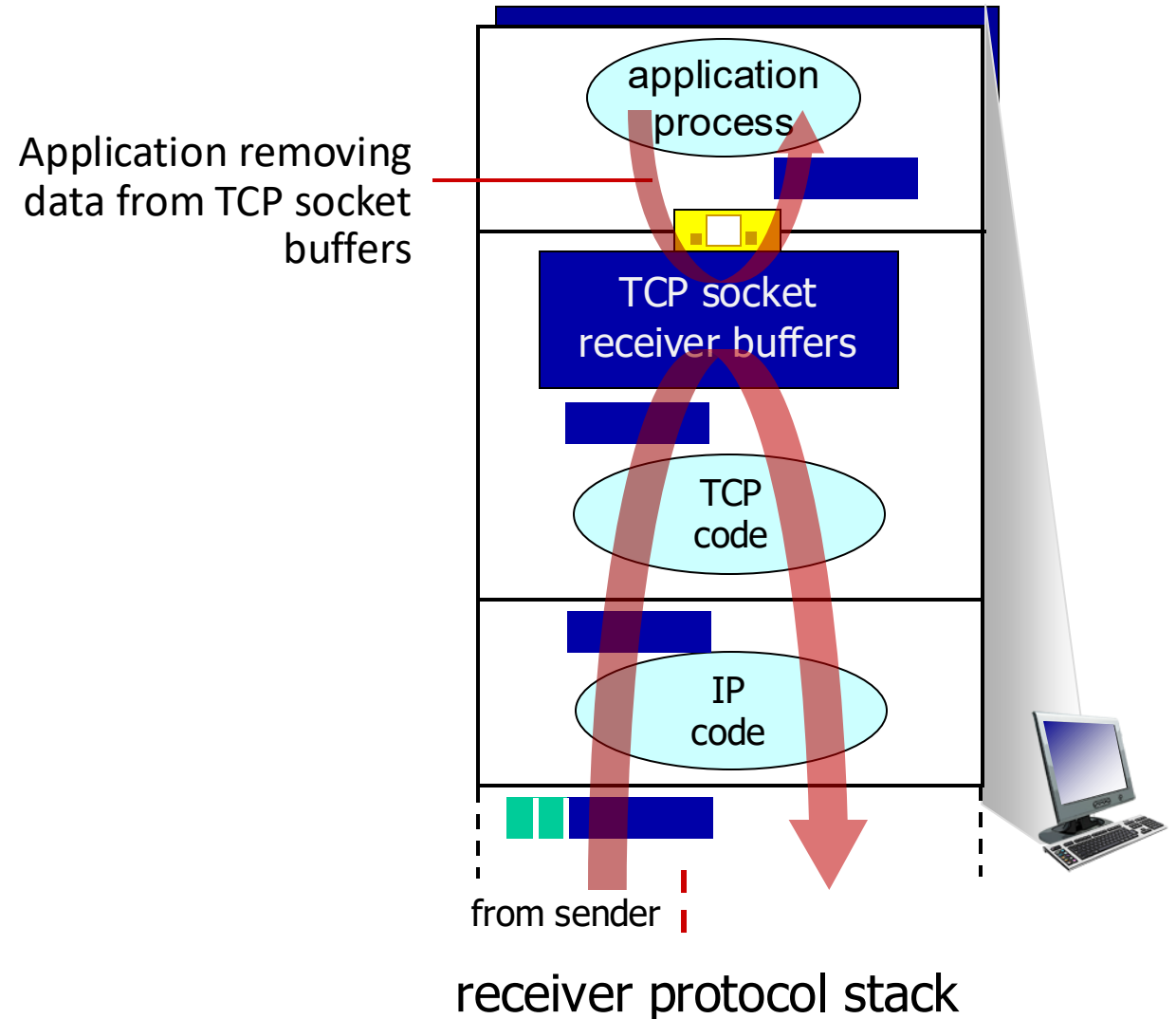


TCP flow control

Q: What happens if network layer delivers data faster than application layer removes data from socket buffers?

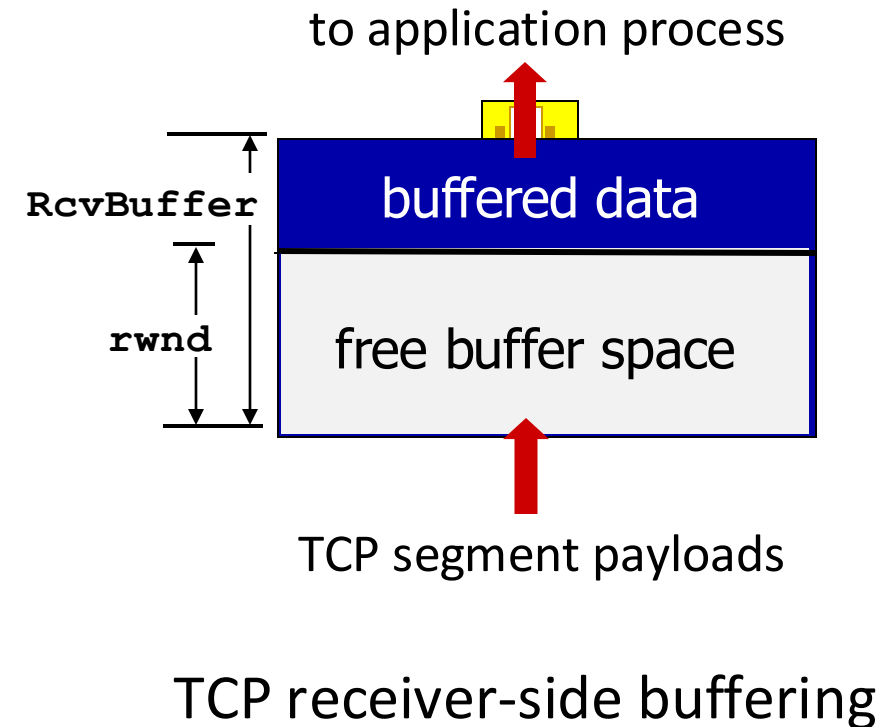
—flow control—

receiver controls sender, so sender won't overflow receiver's buffer by transmitting too much, too fast



TCP flow control

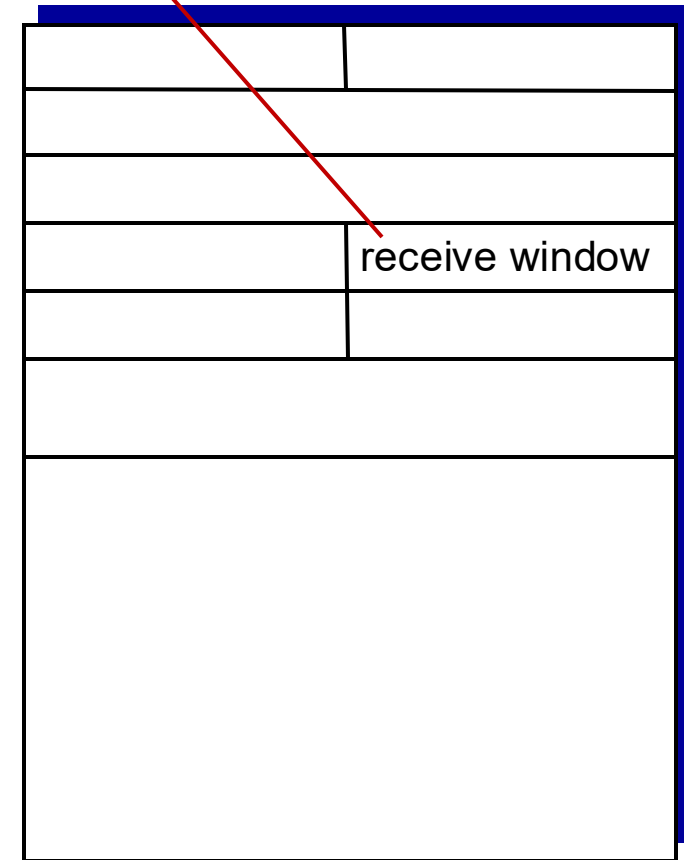
- TCP receiver “advertises” free buffer space in **rwnd** field in TCP header
 - **RcvBuffer** size set via socket options (typical default is **4096 bytes**)
 - many operating systems autoadjust **RcvBuffer**
- sender limits amount of unACKed (“in-flight”) data to received **rwnd**
- guarantees receive buffer will not overflow



TCP flow control

- TCP receiver “advertises” free buffer space in **rwnd** field in TCP header
 - **RcvBuffer** size set via socket options (typical default is 4096 bytes)
 - many operating systems autoadjust **RcvBuffer**
- sender limits amount of unACKed (“in-flight”) data to received **rwnd**
- guarantees receive buffer will not overflow

flow control: # bytes receiver willing to accept



TCP segment format

TCP window scale option

🌐 1 language ▾

Article [Talk](#)

[Read](#) [Edit](#) [View history](#) [Tools](#) ▾

From Wikipedia, the free encyclopedia

The **TCP window scale option** is an option to increase the [receive window](#) size allowed in [Transmission Control Protocol](#) above its former maximum value of 65,535 bytes. This TCP option, along with several others, is defined in [RFC 7323](#) [↗](#) which deals with [long fat networks](#) (LFNs).

TCP windows [\[edit \]](#)

The throughput of a TCP communication is limited by two windows: the [congestion window](#) and the receive window. The congestion window tries not to exceed the capacity of the network ([congestion control](#)); the receive window tries not to exceed the capacity of the receiver to process data ([flow control](#)). The receiver may be overwhelmed by data if for example it is very busy (such as a Web server). Each TCP segment contains the current value of the receive window. If, for example, a sender receives an ack which acknowledges byte 4000 and specifies a receive window of 10000 (bytes), the sender will not send packets after byte 14000, even if the congestion window allows it.

Theory [\[edit \]](#)

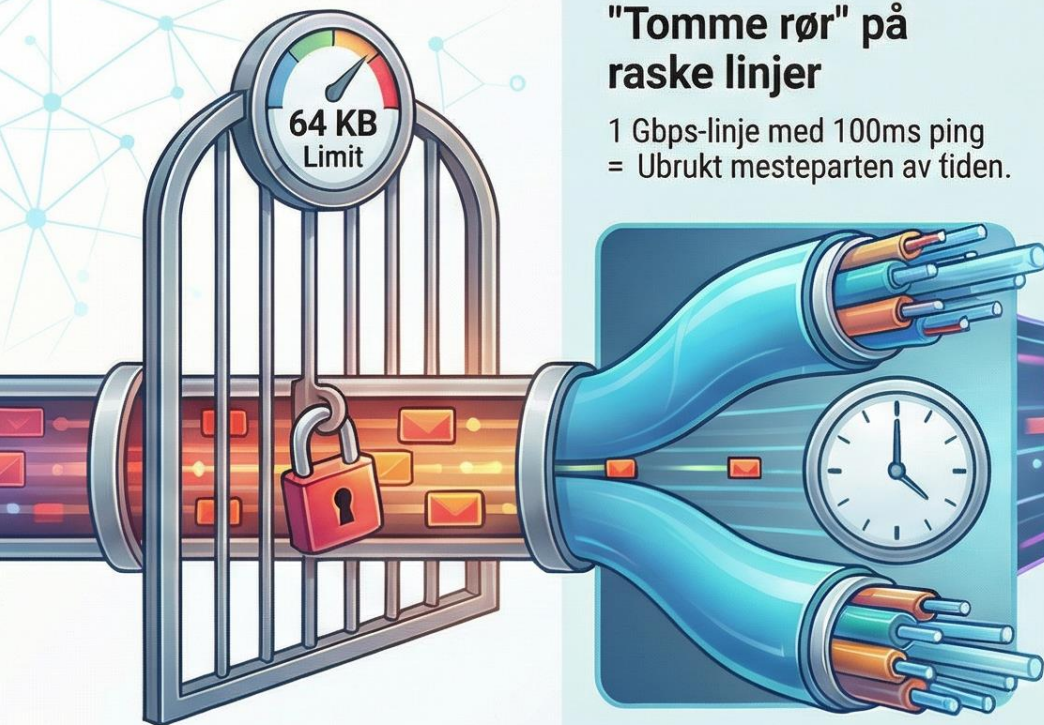
TCP window scale option is needed for efficient transfer of data when the [bandwidth-delay product](#) (BDP) is greater than 64 KB^[1]. For instance, if a [T1](#) transmission line of 1.5 Mbit/second was used over a satellite link with a 513 millisecond [round-trip time](#) (RTT), the bandwidth-delay product is $1,500,000 \times 0.513 = 769,500$ bits or about 96,187 bytes. Using a maximum buffer size of 64 KB^[1] only allows the buffer to be filled to $(65,535 / 96,187) = 68\%$ of the theoretical maximum speed of 1.5 Mbit/second, or 1.02 Mbit/s.

By using the window scale option, the receive window size may be increased up to a maximum value of 1, 073, 725, 440 ($= (2^{16} - 1) \times (2^{14}) = 65,535 \times 16,384$) bytes. This is done by specifying a two byte shift count in the header options field. The true receive window size is left shifted by the value in shift count. A maximum value of 14 may be used for the shift count value. This would allow a single TCP connection to transfer data over the example satellite link at 1.5 Mbit/second utilizing all of the available bandwidth.

Essentially, not more than one full transmission window can be transferred within one round-trip time period. The window scale option enables a single TCP connection to fully utilize an LFN with a BDP of up to 1 GB, e.g. a 10 Gbit/s link with round-trip time of 800 ms.

TCP Window Scale: Slik fjernes "fartsgrensen" på moderne internett

Problemet: 16-bits flaskehalsen



Den magiske grensen på 64 KB

Det originale 16-bits "Receive Window"-feltet tillater maksimalt 65 535 bytes med data i kø.

"Tomme rør" på raske linjer

1 Gbps-linje med 100ms ping
= Ubrukt mesteparten av tiden.

Bandwidth-Delay Product (BDP)

Fenomenet der høy hastighet og forsinkelse begrenses av for små datapakker.

Løsningen: Window Scale (Bit-shifting)

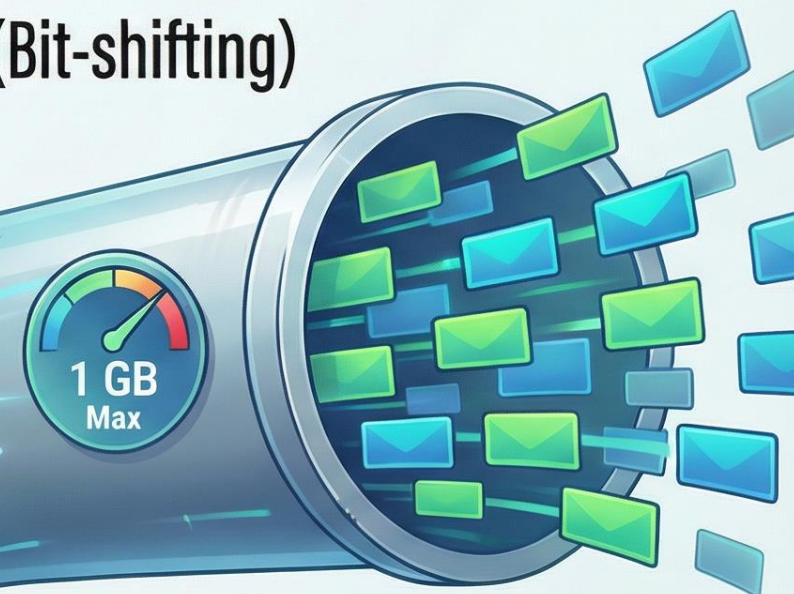
Forhandles under håndtrykket

Skaleringsfaktoren bestemmes kun i de første SYN-pakkene og kan ikke endres senere.



Multiplikator-effekten

Bruker binær "bit-shifting" for å gange vindusstørrelsen med opptil 2^{14} .



Fra 64 KB til 1 GB

Maksimal skalering øker den øvre grensen for datamengde betydelig for å fylle røret.

Vindusstørrelse med vs. uten skalering

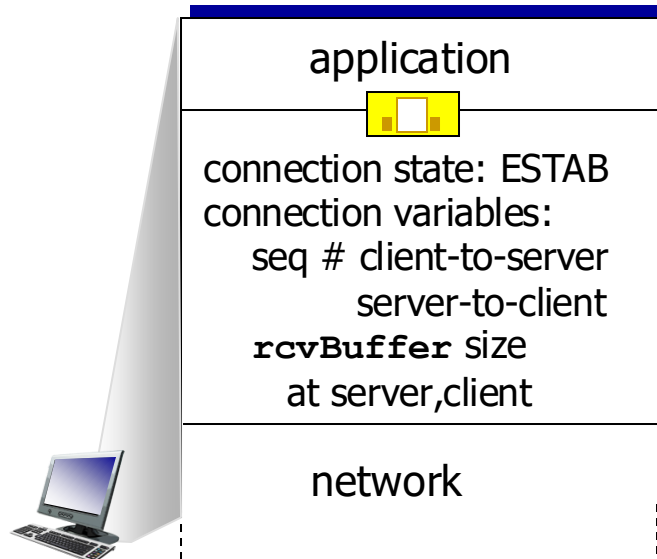
Ingen (0) Multiplikator: 1	65 535 bytes (~64 KB)
Faktor 7 Multiplikator: 128	8 388 480 bytes (~8 MB)
Faktor 14 Multiplikator: 16 384	1 075 725 440 bytes (~1 GB)

Transport Layer: 3-10

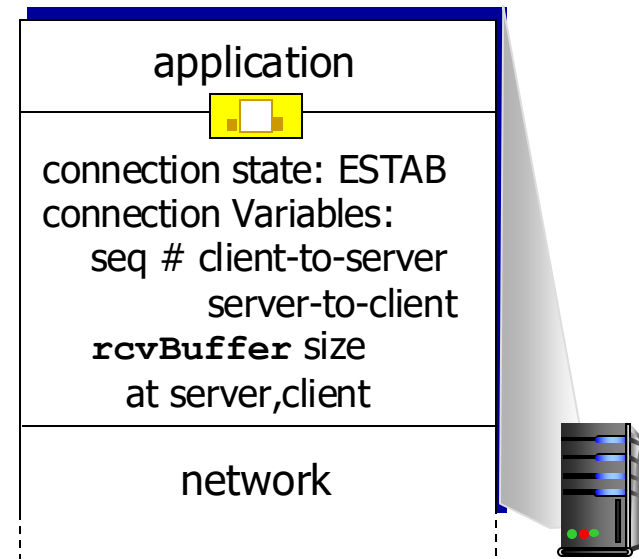
TCP connection management

before exchanging data, sender/receiver “handshake”:

- agree to establish connection (each knowing the other willing to establish connection)
- agree on connection parameters (e.g., starting seq #s)



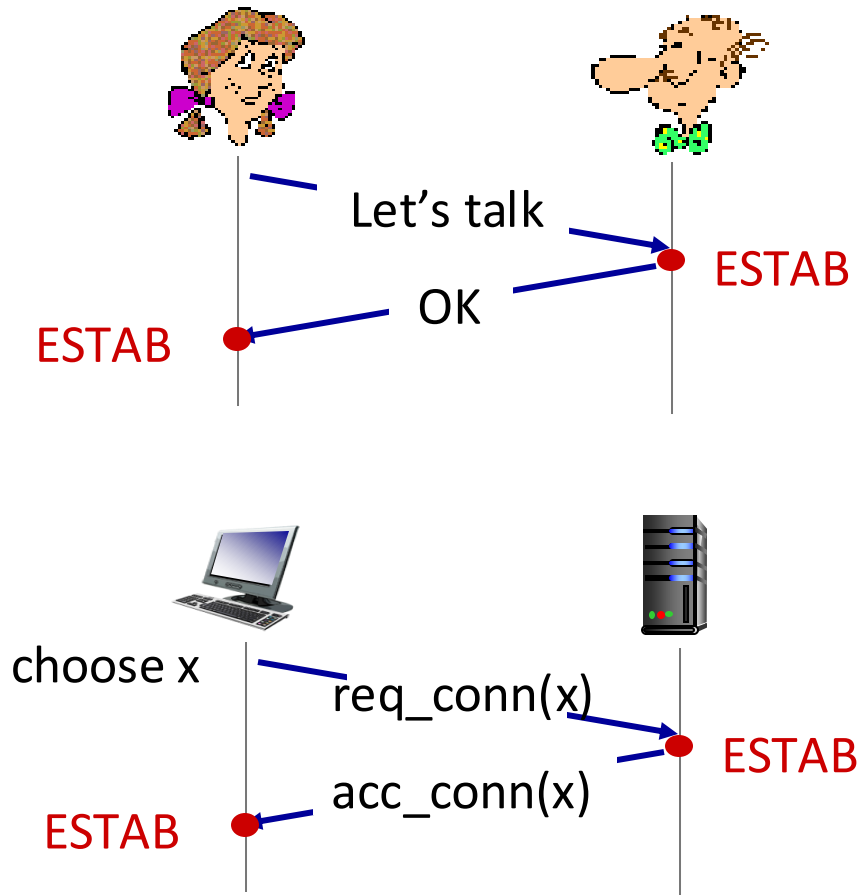
```
Socket clientSocket =  
    newSocket("hostname", "port number");
```



```
Socket connectionSocket =  
    welcomeSocket.accept();
```

Agreeing to establish a connection

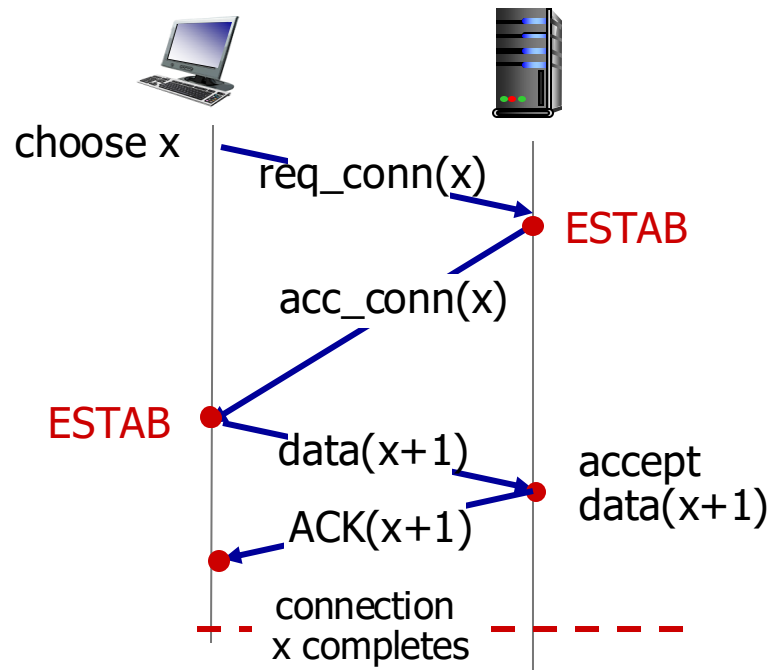
2-way handshake:



Q: will 2-way handshake always work in network?

- variable delays
- retransmitted messages (e.g. req_conn(x)) due to message loss
- message reordering
- can't "see" other side

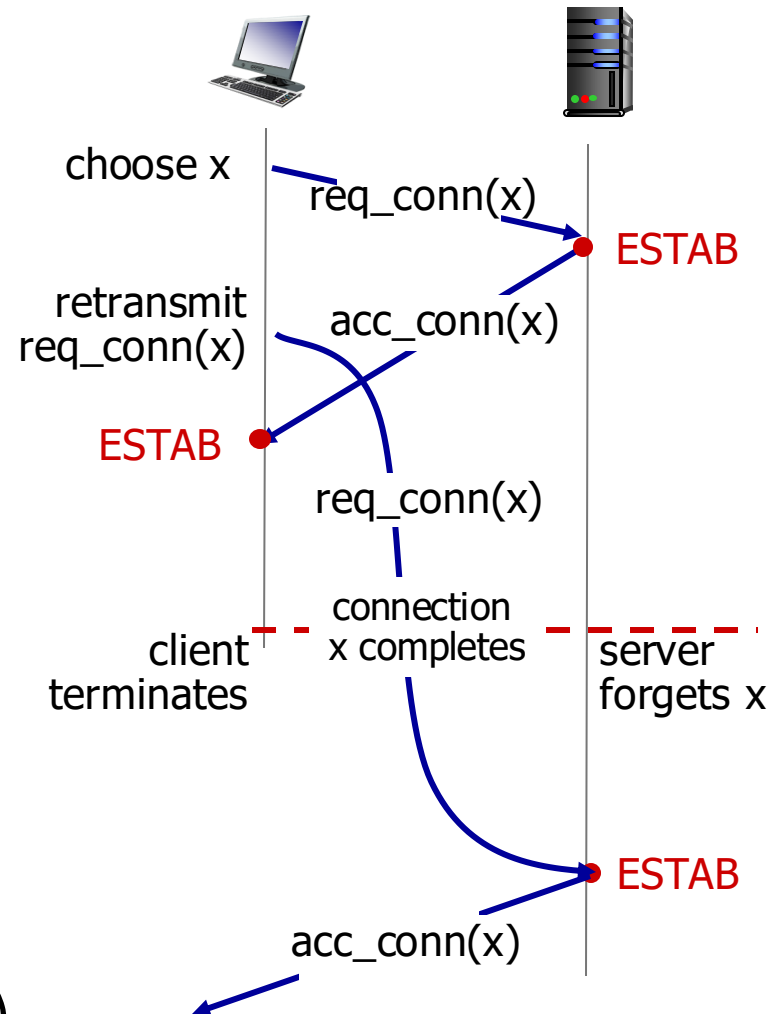
2-way handshake scenarios



No problem!

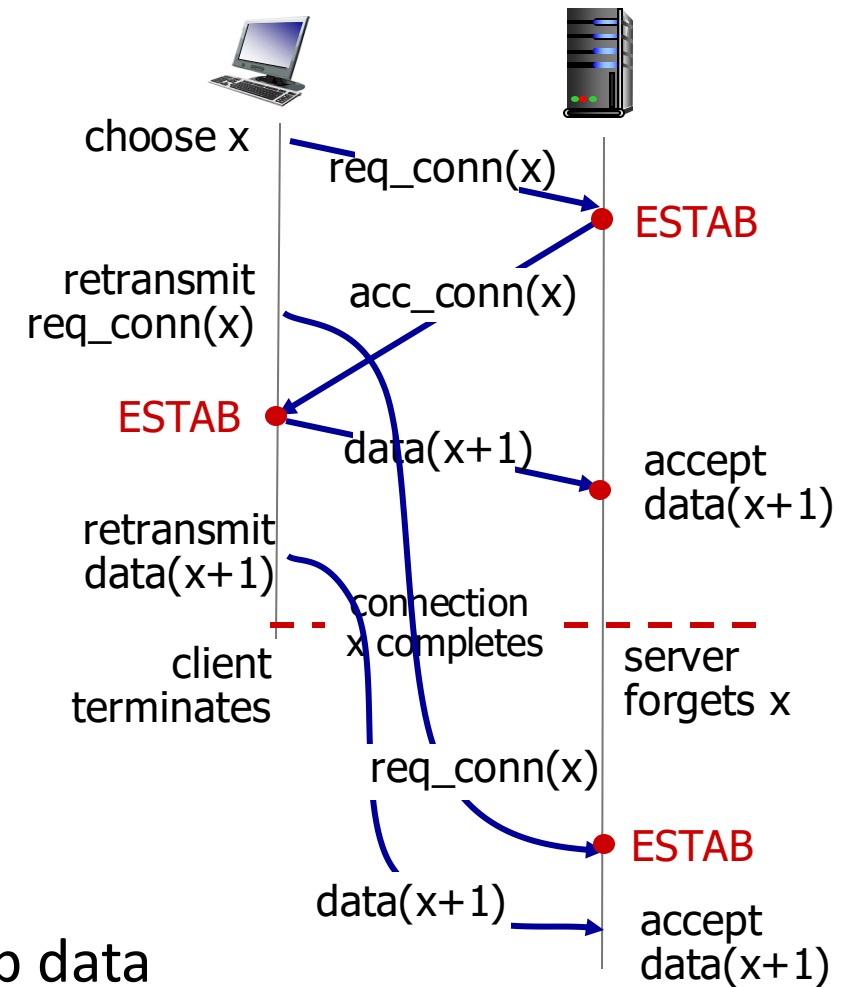


2-way handshake scenarios



Problem: half open connection! (no client)

2-way handshake scenarios



Problem: dup data
accepted!

TCP 3-way handshake

Client state

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```

LISTEN

```
clientSocket.connect((serverName, serverPort))
```

SYNSENT

ESTAB

choose init seq num, x
send TCP SYN msg

received SYNACK(x)
indicates server is live;
send ACK for SYNACK;
this segment may contain
client-to-server data



SYNbit=1, Seq=x

SYNbit=1, Seq=y
ACKbit=1; ACKnum=x+1

ACKbit=1, ACKnum=y+1



choose init seq num, y
send TCP SYNACK
msg, acking SYN

received ACK(y)
indicates client is live

Server state

```
serverSocket = socket(AF_INET, SOCK_STREAM)  
serverSocket.bind('', serverPort)  
serverSocket.listen(1)  
connectionSocket, addr = serverSocket.accept()
```

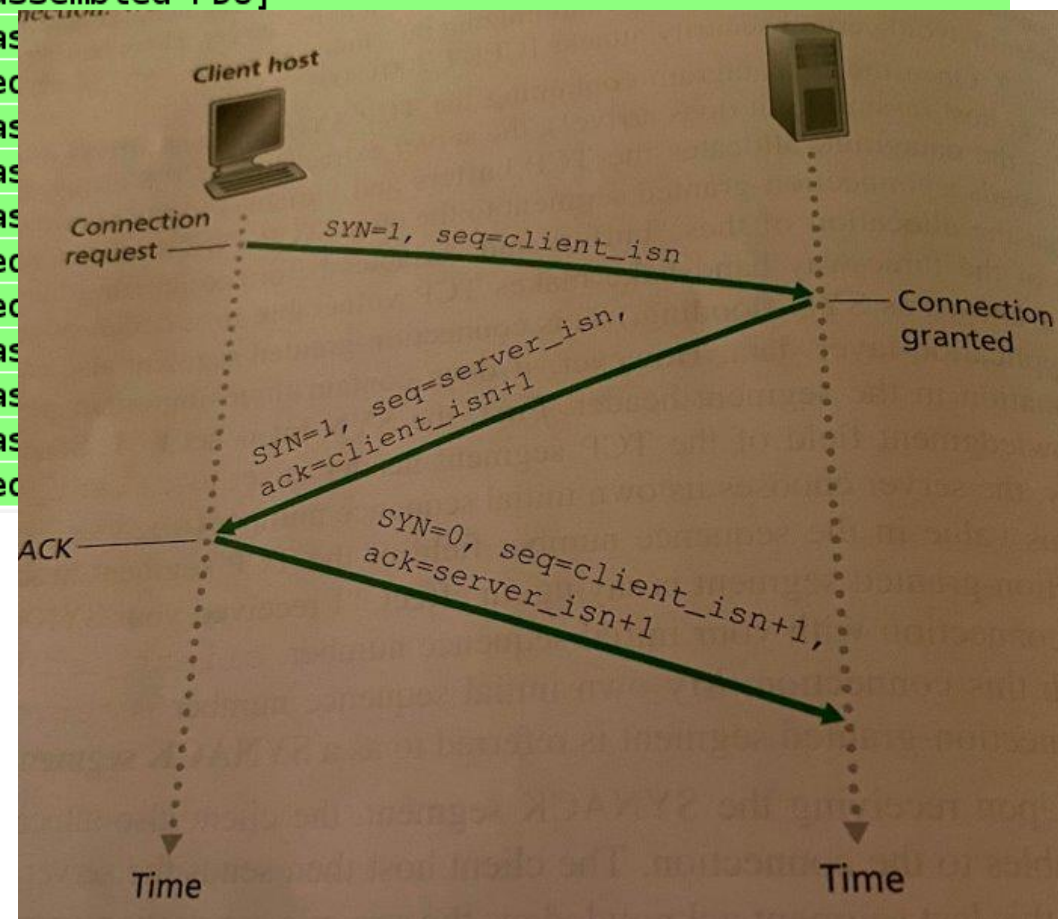
LISTEN

SYN RCVD

ESTAB

TCP connection set-up in wireshark

Source	Destination	Protocol	Info
172.16.3.226	158.39.11.240	TCP	61797 > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=3 TSV=739718215 T
158.39.11.240	172.16.3.226	TCP	http > 61797 [SYN, ACK] Seq=0 Ack=1 Win=16384 Len=0 MSS=1460 WS=0 TSV=
172.16.3.226	158.39.11.240	TCP	61797 > http [ACK] Seq=1 Ack=1 Win=524280 Len=0 TSV=739718216 TSER=0
172.16.3.226	158.39.11.240	HTTP	GET /00_ROOT/links.htm HTTP/1.1
158.39.11.240	172.16.3.226	TCP	[TCP segment of a reassembled PDU]
158.39.11.240	172.16.3.226	TCP	[TCP segment of a reassembled PDU]
172.16.3.226	158.39.11.240	TCP	61797 > http [ACK] Seq=
158.39.11.240	172.16.3.226	TCP	[TCP segment of a reassembled PDU]
158.39.11.240	172.16.3.226	TCP	[TCP segment of a reassembled PDU]
158.39.11.240	172.16.3.226	TCP	[TCP segment of a reassembled PDU]
158.39.11.240	172.16.3.226	TCP	[TCP segment of a reassembled PDU]
172.16.3.226	158.39.11.240	TCP	61797 > http [ACK] Seq=
172.16.3.226	158.39.11.240	TCP	61797 > http [ACK] Seq=
158.39.11.240	172.16.3.226	TCP	[TCP segment of a reassembled PDU]
158.39.11.240	172.16.3.226	TCP	[TCP segment of a reassembled PDU]
158.39.11.240	172.16.3.226	TCP	[TCP segment of a reassembled PDU]
172.16.3.226	158.39.11.240	TCP	61797 > http [ACK] Seq=

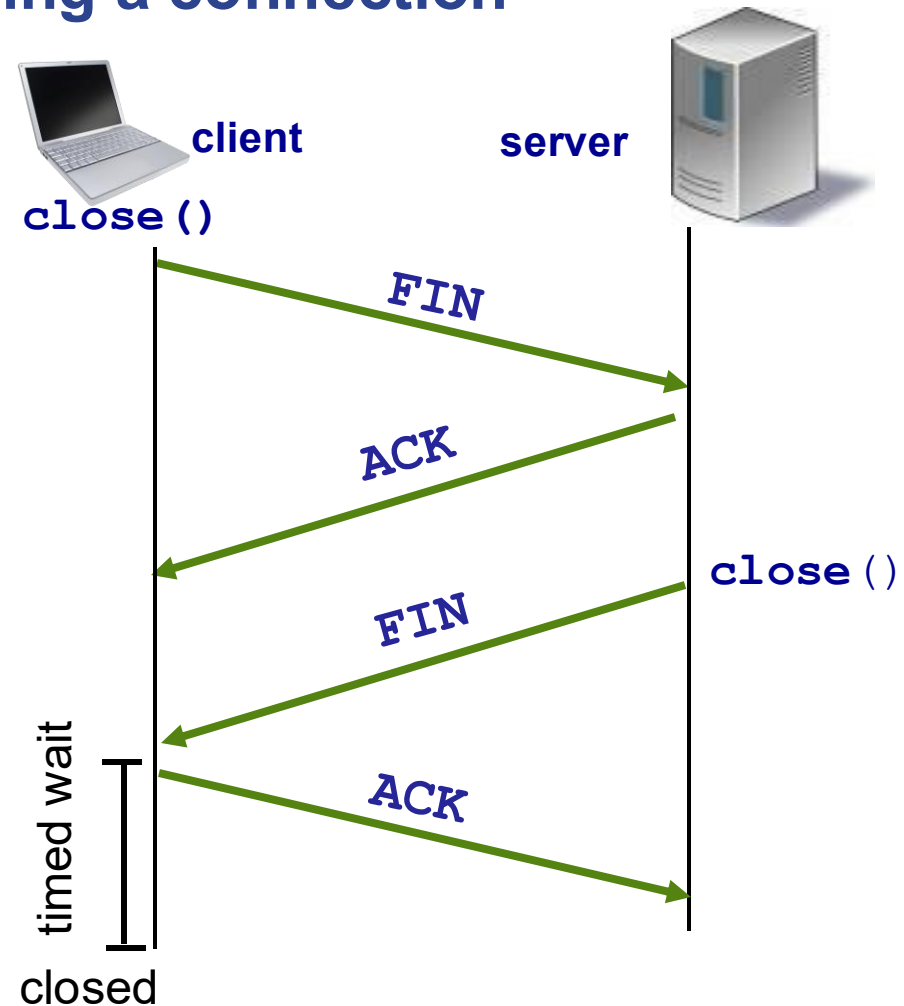


Closing a TCP connection

- client, server each close their side of connection
 - send TCP segment with FIN bit = 1
- respond to received FIN with ACK
 - on receiving FIN, ACK can be combined with own FIN
- simultaneous FIN exchanges can be handled

TCP connection management – closing a connection

- Client closes socket
`clientSocket.close();`
FIN
- Server returns **ACK**
- Server closes connection, **FIN**
- Client sends **ACK** enters TIME_WAIT state to be able to resend **ACK** to received **FINs**
- Connection closed at server when **ACK** received



Time	Source	Destination	Protocol	Length	Info
122.260653	10.0.0.103	193.213.115.9	TCP	54	49441 → pop3s [FIN, ACK] Seq=795 Ack=1824 Win=65535 Len=0
122.266248	193.213.115.9	10.0.0.103	TCP	54	pop3s → 49441 [ACK] Seq=1824 Ack=796 win=49640 Len=0
122.266322	10.0.0.103	193.213.115.9	TCP	54	[TCP Dup ACK 1110#1] 49441 → pop3s [ACK] Seq=796 Ack=1824 Win=65535 Len=0
122.266409	193.213.115.9	10.0.0.103	TCP	54	pop3s → 49441 [FIN, ACK] Seq=1824 Ack=796 Win=49640 Len=0
122.266440	10.0.0.103	193.213.115.9	TCP	54	49441 → pop3s [ACK] Seq=796 Ack=1825 Win=65535 Len=0

TCP states visited by ClientSide

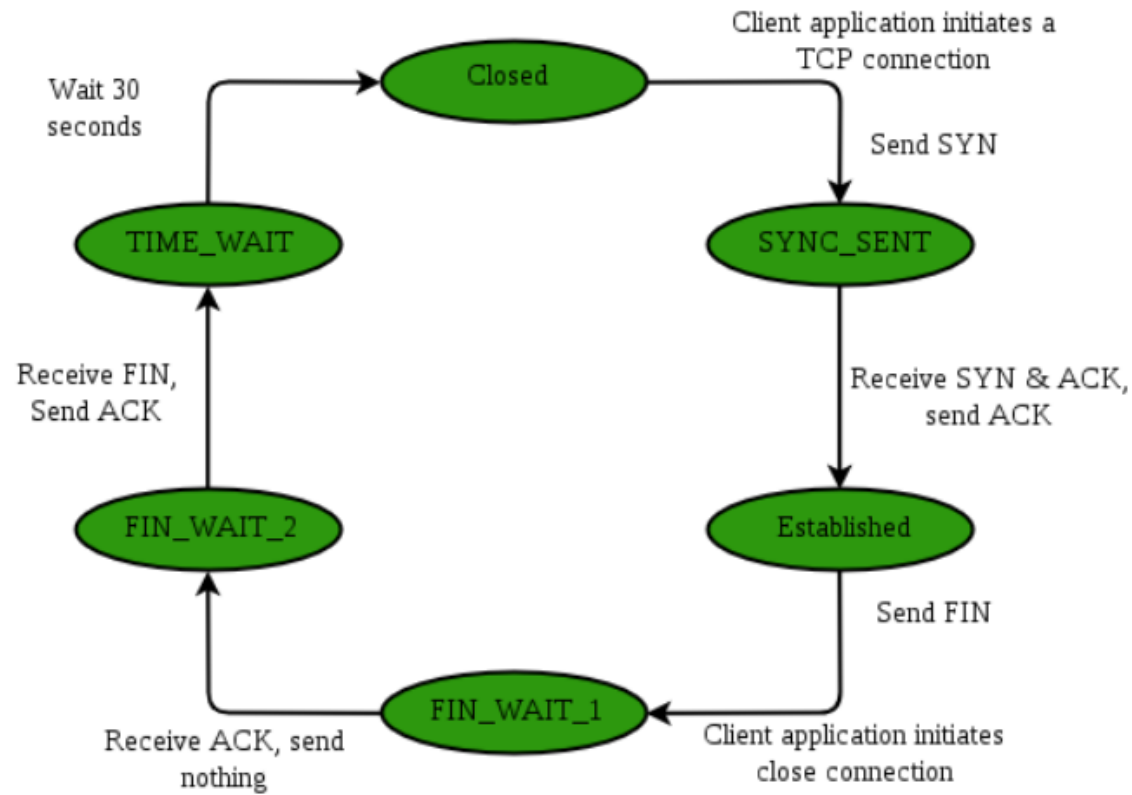


Fig : TCP states visited by a client TCP

TCP states visited by ServerSide

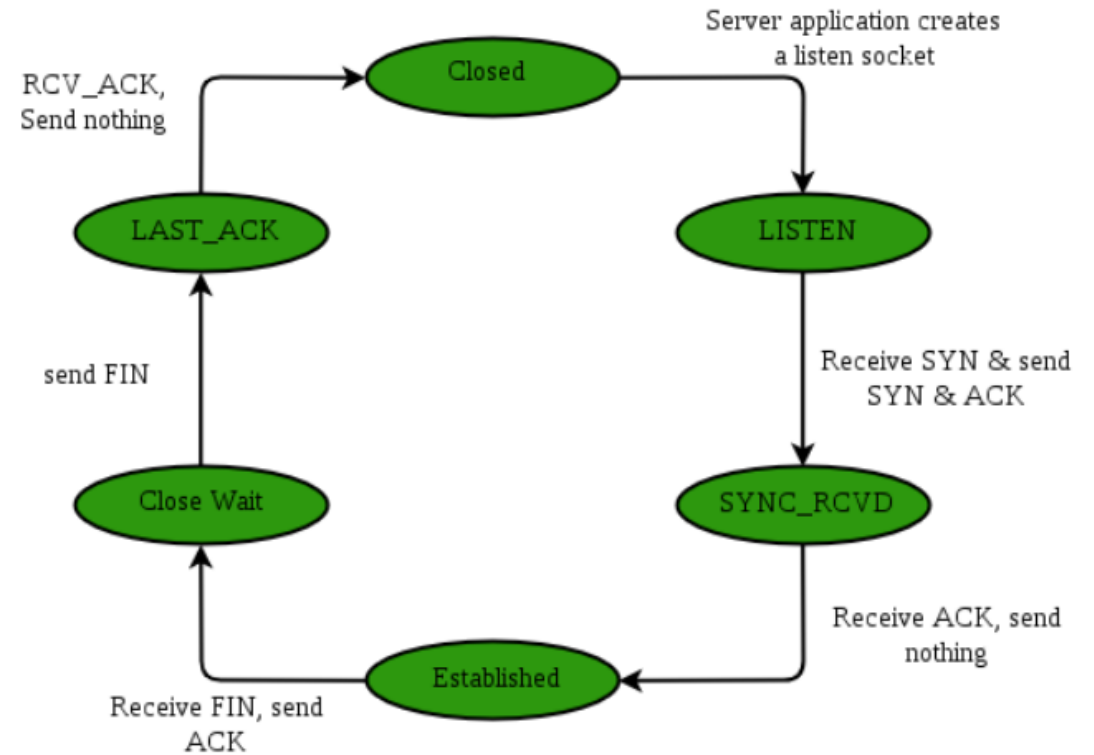
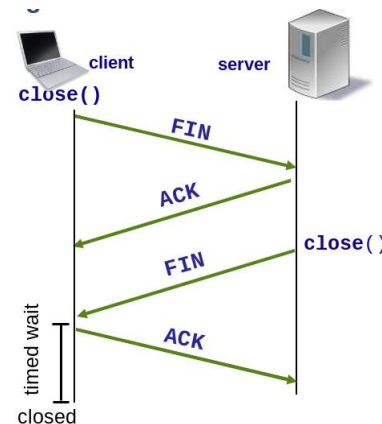


Fig : TCP states visited by a client TCP



Chapter 3: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- **Principles of congestion control**
- TCP congestion control
- Evolution of transport-layer functionality



Principles of congestion control

Congestion:

- informally: “too many sources sending too much data too fast for *network* to handle”
- manifestations:
 - long delays (queueing in router buffers)
 - packet loss (buffer overflow at routers)
- different from flow control!
- a top-10 problem!



congestion control:

too many senders,
sending too fast

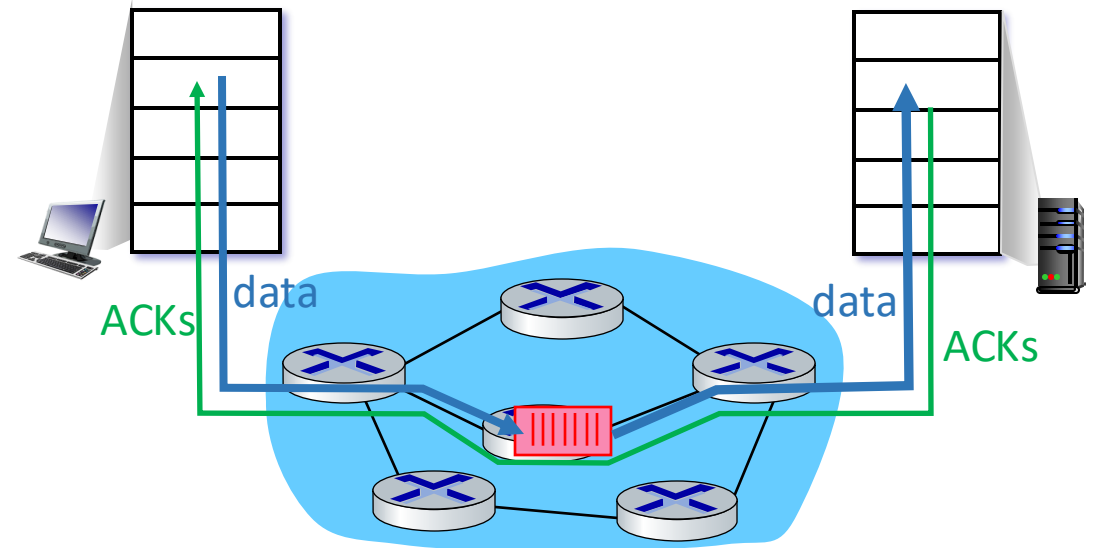


flow control: one sender
too fast for one receiver

Approaches towards congestion control

End-end congestion control:

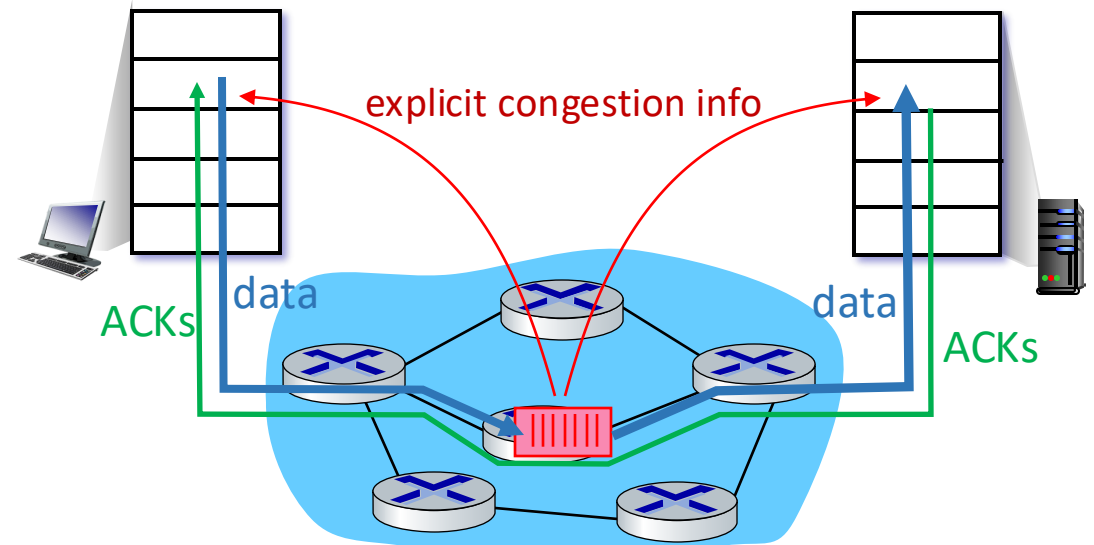
- no explicit feedback from network
- congestion *inferred* from observed loss, delay
- approach taken by TCP



Approaches towards congestion control

Network-assisted congestion control:

- routers provide *direct* feedback to sending/receiving hosts with flows passing through congested router
- may indicate congestion level or explicitly set sending rate
- **TCP ECN**, ATM, DECbit protocols



What is the ECN field in the IP header?

ECN field in the differentiated services field of the IP packet header. The ECN field combinations "01" and "10" are set by the data sender to indicate that the end-points of the transport protocol are ECN capable. The non ECT combination "00" indicates a packet that is not using ECN.

Chapter 3: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- Principles of congestion control
- **TCP congestion control**
- **Evolution of transport-layer functionality**

